

Lab 11: Memory Management, Garbage Collection, and Runtime Systems

1. Objectives

- Understand how memory is managed during program execution
- Learn the difference between stack and heap memory
- Understand the concept of runtime systems
- Observe how garbage collection works in managed languages
- Demonstrate garbage collection behavior using Java or Python.

2. Memory Management

Memory management refers to the process of allocating, using, and releasing memory during program execution. Every program requires memory to store variables, objects, and intermediate results. Efficient memory management ensures that programs run correctly without wasting memory resources.

Modern programming languages manage memory in different ways. Some languages require programmers to manually allocate and free memory, while others use automatic memory management techniques such as garbage collection.

Two important memory regions used during program execution are **stack memory** and **heap memory**.

2.1. Stack Memory

Stack memory is used to store method calls, local variables, and function parameters. It follows a simple principle called Last In, First Out. This means the last value added to the stack is the first one removed.

When a method is called, its variables and execution details are pushed onto the stack. Once the method finishes execution, all the memory used by that method is automatically removed. Because of this behavior, stack memory is very fast and efficient. However, stack memory is limited in size, which is why it is mainly used for short-lived data.

In simple terms, stack memory is responsible for managing the execution flow of a program.

```
#Python
def add(a, b):
    result = a + b
    return result

x = 10
y = 5

print(add(x, y))
```

```
// Java
public class StackExample {

    static int add(int a, int b){
        int result = a + b;
        return result;
    }

    public static void main(String[] args){
        int x = 10;
        int y = 5;

        System.out.println(add(x, y));
    }
}
```

Result Explanation

- Variables `x` and `y` are stored in stack memory.
- When the function `add()` is called, a new stack frame is created.
- The local variable `result` exists only inside that function.
- After the function finishes execution, the stack frame is removed.

2.2. Heap Memory

Heap memory is used to store objects and dynamically allocated data. Unlike stack memory, heap memory is much larger and is shared across the entire application. Objects created during program execution remain in heap memory until they are no longer needed.

Heap memory allocation is slower compared to stack memory, but it allows more flexibility. Heap is especially important in Object-Oriented Programming, where objects need to live beyond the scope of a single method.

Because heap memory does not automatically free unused objects, another mechanism is needed to manage it efficiently. This is where Garbage Collection comes in.

```
#Python
class Student:
    def __init__(self, name):
        self.name = name

s1 = Student("Kamal")
s2 = Student("Nimal")

print(s1.name)
```

```
// Java
class Student {

    String name;

    Student(String name){
        this.name = name;
    }

    public static void main(String[] args){

        Student s1 = new Student("Kamal");
        Student s2 = new Student("Nimal");

        System.out.println(s1.name);
    }
}
```

Result Explanation

- Objects `s1` and `s2` are stored in heap memory.
- Variables `s1` and `s2` store references to those objects.
- When references are removed, objects become eligible for garbage collection.

3. Garbage Collection

Garbage Collection is an automatic memory management process that runs in the background. Its main task is to identify objects in heap memory that are no longer used by the program and remove them to free up memory.

An object becomes eligible for garbage collection when there are no active references pointing to it. Once detected, the garbage collector frees the memory occupied by that object and makes it available for future use. This helps prevent memory leaks and improves application performance.

Advantages of garbage collection:

- Prevents memory leaks
- Reduces programmer errors
- Automatically manages unused memory

An object becomes eligible for garbage collection when **no references point to it**.

3.1. Garbage Collection in Python

Garbage Collection in Python is an automatic process that handles memory allocation and deallocation, ensuring efficient use of memory. Python automatically manages memory through two primary strategies:

1. Reference counting
2. Garbage collection

3.1.1. Reference counting

Python uses reference counting to manage memory. Each object keeps track of how many references point to it. When the reference count drops to zero i.e., no references remain, Python automatically deallocates the object.

```
#Python
import sys

x = [1, 2, 3]
print(sys.getrefcount(x))

y = x
print(sys.getrefcount(x))

y = None
print(sys.getrefcount(x))
```

Explanation:

- `x` initially has one reference from the variable `x`. When `sys.getrefcount(x)` is called, Python temporarily creates another reference while passing `x` to the function.
- Because of this, the returned value is always one more than the actual reference count.
- Assigning `y = x` increases the reference count because both `x` and `y` refer to the same object.
- Setting `y = None` removes its reference, decreasing the count again.

3.1.2. Garbage collection for Cyclic References

Reference counting fails in the presence of cyclic references i.e., objects that reference each other in a cycle. Even if nothing else points to them, their reference count never reaches zero.

```
#Python
import sys
x = [1, 2, 3]
y = [4, 5, 6]

x.append(y)
y.append(x)
print(sys.getrefcount(x))
print(sys.getrefcount(y))
```

Explanation:

x contains y and y contains x.

Even after deleting x and y, Python won't be able to free the memory just using reference counting, because each still references the other.

To handle such circular references, Python uses a Garbage Collector (GC) from the built-in `gc` module. This collector is able to detect and clean up objects involved in reference cycles.

Generational Garbage Collection

Python's Generational Garbage Collector is designed to deal with cyclic references. It organizes objects into three generations based on their lifespan:

Generation 0: Newly created objects.

Generation 1: Objects that survived one collection cycle.

Generation 2: Long-lived objects.

When reference cycles occur, the garbage collector automatically detects and cleans them up, freeing the memory.

Automatic Garbage Collection of Cycles

Garbage collection runs automatically when the number of allocations exceeds the number of deallocations by a certain threshold. This threshold can be inspected using the `gc` module.

```
#Python
import gc
print(gc.get_threshold())
```

Explanation: It returns the threshold tuple for generations 0, 1 and 2. When allocations exceed the threshold, collection is triggered.

Manual garbage collection

Sometimes it's beneficial to manually invoke the garbage collector, especially in the case of reference cycles.

```
#Python
import gc

# Create a cycle
def fun(i):
    x = {}
    x[i + 1] = x
    return x

# Trigger garbage collection
c = gc.collect()
print(c)

for i in range(10):
    fun(i)

c = gc.collect()
print(c)
```

Explanation:

- def fun(i) creates a cyclic reference by making a dictionary reference itself.
- gc.collect() triggers garbage collection and stores the count of collected objects (initially 0).
- for i in range(10) calls fun(i) 10 times, creating 10 cyclic references.
- gc.collect() triggers garbage collection again and prints the count of collected cycles

3.2. Garbage collection in Java

Garbage Collection in Java is an automatic memory management process that removes objects from heap memory when they are no longer being used by the program. The Java Virtual Machine (JVM) automatically identifies unused objects and reclaims their memory.

An object becomes eligible for garbage collection when no references point to it. Once the JVM detects such objects, the garbage collector removes them and frees the memory so that it can be reused for new objects.

An object becomes eligible for garbage collection in several situations:

- When the reference to the object is set to null
- When the reference variable is reassigned to another object
- When the object is created inside a method and the method finishes execution
- When two objects reference each other but are no longer accessible from the program

```
// Java
public class GarbageExample1 {

    public static void main(String[] args) {

        Student s1 = new Student("Kamal");

        s1 = null;    // Object becomes eligible for garbage collection

        System.gc(); // Request JVM to run garbage collector

        System.out.println("Garbage collection requested");
    }
}

class Student {

    String name;

    Student(String name) {
        this.name = name;
    }
}
```

Explanation

- An object of the `Student` class is created in heap memory.
- The variable `s1` holds the reference to that object.
- When `s1` is set to `null`, the object no longer has any references.
- Therefore, it becomes **eligible for garbage collection**.
- `System.gc()` requests the JVM to run the garbage collector (but execution is not guaranteed immediately).

```
// Java
public class GarbageExample2 {

    public static void main(String[] args) {

        Student s1 = new Student("Kamal");
        Student s2 = new Student("Nimal");
```

```
s1 = s2; // The first object becomes unreachable

System.gc();

System.out.println("Garbage collection may occur");
}
}

class Student {

    String name;

    Student(String name) {
        this.name = name;
    }
}
```

Explanation

- Two objects are created in heap memory.
- `s1` initially refers to the first object and `s2` refers to the second.
- When `s1 = s2` is executed, the reference to the first object is lost.
- Since no variables refer to the first object anymore, it becomes **eligible for garbage collection**.

4. Lab Tasks

Task 01

Write a Python program that:

- Creates multiple objects of a class
- Prints a message when objects are created
- Deletes one of the objects using `del`
- Forces garbage collection using `gc.collect()`

Explain the output observed when the object is deleted.

Task 02

Write a Python program that:

- Creates a list containing several objects
- Removes references to some objects
- Calls the garbage collector

Observe and explain when the objects are removed from memory.

Task 03

Write a Java program that:

- Creates at least three objects
- Removes references to one or two objects using `null`
- Calls `System.gc()`

Explain whether garbage collection occurs immediately and why.

Task 04

Write a java program that demonstrates the difference between **stack variables and heap objects**.

Requirements:

- Create a function with local variables
- Create an object inside the function
- Print messages to show when variables are created and destroyed

Explain which variables are stored in stack and which are stored in heap.

5. Submission

Create a PDF file containing screenshots of all your codes and Results along with the explanations. Rename the PDF file as **CO523_Lab11_EXXYYY.pdf**, where EXXYYY represents your E-number.

Upload the file to the FEeLS by the given deadline.